重心
直径
$Dijkstra$(单源最短路径)
$SPFA$(用于负环判断，差分约束)
Prim(最小生成树)
$Kurskal$
Kruskal重构树
缩点SCC
2-SAT
边双连通分量E-DCC
点双连通分量V-DCC
割点
割边
二分图判断(判奇环)
最大二分图匹配
最大流
最小费用最大流
欧拉回路/路径
点分治
树上启发式合并

图论

重心

```
void solve()
{
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>siz(n + 1),fl(n + 1),cg;
    auto dfs = [&](auto &&self,int u,int f) -> void
    {
        siz[u] = 1;
        int mx = 0;
        for(int v : g[u])
        {
            if(v == f)continue;
            self(self,v,u);
            mx = std::max(mx,siz[v]);
            siz[u] += siz[v];
        }
```

```
        mx = std::max(mx,n - siz[u]);
        fl[u] = mx <= (n >> 1);
        if(fl[u])cg.push_back(u);
    };
    dfs(dfs,1,0);
}
```

直径

```
void solve()
{
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>dep(n + 1);
    int d1,d2;
    auto dfs = [&](auto &&self,int u,int f,int &mx) -> void
    {
        if(dep[u] > dep[mx])mx = u;
        for(int v : g[u])
        {
            if(v == f)continue;
            dep[v] = dep[u] + 1;
            self(self,v,u,mx);
        }
    };
    d1 = 1,dep[1] = 0;
    dfs(dfs,1,0,d1);
    d2 = d1,dep[d1] = 0;
    dfs(dfs,d1,0,d2);
}
```

Dijkstra(单源最短路径)

```
void dijkstra(int st)
{
    int n;
    vector<int>dis(n + 1,INF);
    vector<bool>vis(n + 1);
    dis[st] = 0;
    priority_queue<PA>q;
    q.push({-dis[st],st});
```

```
    while(!q.empty())
    {
        auto [d,u] = q.top();q.pop();
        if(vis[u])continue;
        vis[u] = 1;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(vis[v])continue;
            if(dis[v] > dis[u] + e[i].w)
            {
                dis[v] = dis[u] + e[i].w;
                q.push({dis[v],v});
            }
        }
    }
}
```

$SPFA$(用于负环判断，差分约束)

```
bool SPFA()
{
    while(!q.empty())
    {
        int u = q.front();
        q.pop();
        vis[u] = 0;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(dis[u] + e[i].w < dis[v])
            {
                dis[v] = dis[u] + e[i].w;
                if(!vis[v])
                {
                    q.push(v);
                    vis[v] = 1;
                    //计算最短路时可不需要下面
                    cnt[v]++;
                    if(cnt[v]>n)return true;
                }
            }
        }
    }
    return false;
}
//差分约束建边
求解一个变量序列最大值或最小值，可通过建立对应差分约束系统
求解最短路径或者最长路径来实现,其最短路或最长路就是这个变量可能的最大值或最小值
最短路径i + k >= j,add(i,j,k);
最长路径i + k <= j,add(i,j,k);
```

Prim(最小生成树)

```
void prim()
{
    int tot = 0,ans = 0;
    priority_queue<PA>q;
    vector<int>dis(n + 1,INF);
    vector<bool>vis(n + 1);
    dis[1] = 0;
    q.push({-dis[1],1});
    while(!q.empty())
    {
        auto [d,u] = q.top();q.pop();
        d = -d;
        if(d != dis[u])continue;
        vis[u] = 1;
        ans += dis[u];
        tot++;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(!vis[v] && dis[v] > e[i].w)
            {
                dis[v] = e[i].w;
                q.push({v,dis[v]});
            }
        }
    }
    if(tot != n)ans = -1;//无法生成一颗树
}
```

Kurskal

```
void kruskal()
{
    sort(edge,edge + m,cmp);
    for(re int i = 0;i < m;i++)
    {
        eu = find(edge[i].u), ev = find(edge[i].v);
        if(eu == ev)
        {
            continue;
        }
        ans += edge[i].w;
        fa[ev] = eu;
        if(++cnt == n - 1)
        {
            break;
        }
    }
}
```

Kruskal重构树

```
std::vector<std::array<int,3>>edge(m);
std::sort(edge.begin(),edge.end());
std::vector g(2 * n ,std::vector<int>());
std::vector<int>w(2 * n);
dsu ds = dsu(2 * n);//并查集
int tot = n;
for(auto [w,u,v] : edge)
{
    int pu = ds.find(u);
    int pv = ds.find(v);
    if(pu == pv)continue;
    ds.fa[pu] = ++tot;
    ds.fa[pv] = tot;
    w[tot] = w;
    g[tot].push_back(pu);
    g[tot].push_back(pv);
}
```

缩点SCC

```
int low[N],dfn[N],times = 0;
int belong[N],tot,dis[N];//缩点后每个点的归属，缩点后数量,缩点的权值
bool vis[N];//当前点是否正在被缩
stack<int>st;
void tarjan(int p)
{
    low[p] = dfn[p] = ++times;
    vis[p] = 1;
    st.push(p);
    for (int i = heads[p];i; i = e[i].nx)
    {
        int v = e[i].to;
        if (!dfn[v])
        {
            tarjan(v);
            low[p] = min(low[p], low[v]);
        }
        else if (vis[v])
            low[p] = min(low[p], dfn[v]);
    }
    if (low[p] == dfn[p])
    {
        tot++;
        while (1)
        {
            int x = st.top();
            st.pop();
            belong[x] = tot;
```

```
            vis[x] = 0;
            if (x == p)break;
        }
    }
}
```

2-SAT

```
int n,m;
std::cin >> n >> m;
std::vector g(n * 2 + 1,std::vector<int>());
for(int i = 1;i <= m;i++)
{
    int u,op1,v,op2;
    std::cin >> u >> op1 >> v >> op2;
    g[!op1 * n + u].push_back(op2 * n + v);
    g[!op2 * n + v].push_back(op1 * n + u);
}
std::vector<int>dfn(2 * n + 1),low(2 * n + 1),vis(2 * n + 1),bel(2 * n + 1);
std::stack<int>stk;
int times = 0,tot = 0;
auto dfs = [&](auto &&self,int u) -> void
{
    vis[u] = 1;
    stk.push(u);
    dfn[u] = low[u] = ++times;
    for(int v : g[u])
    {
        if(!dfn[v])
        {
            self(self,v);
            low[u] = std::min(low[u],low[v]);
        }
        else if(vis[v])low[u] = std::min(low[u],dfn[v]);
    }
    if(dfn[u] == low[u])
    {
        tot++;
        while(!stk.empty())
        {
            int x = stk.top();stk.pop();
            bel[x] = tot;
            vis[x] = 0;
            if(x == u)break;
        }
    }
};
for(int i = 1;i <= 2 * n;i++)
    if(!dfn[i])dfs(dfs,i);

for(int i = 1;i <= n;i++)
    if(bel[i] == bel[i + n])
```

```
    {
        std::cout << "IMPOSSIBLE\n";
        return;
    }
std::cout << "POSSIBLE\n";
for(int i = 1;i <= n;i++)
    std::cout << (bel[i + n] < bel[i]) << " \n"[i == n];
```

边双连通分量E-DCC

```
struct edge
{
    int nx;
    int to;
}e[M];int heads[N],cntqq = 1;
void add(int u,int v)
{
    e[++cntqq] = {heads[u],v};
    heads[u] = cntqq;
}
void solve()
{
    //初始化
    cntqq = 1;
    memset(heads,0,sizeof(int) * (n + 1));

    std::vector<int>low(n + 1),dfn(n + 1),bel(n + 1);//属于哪个缩点
    std::vector<bool>bz(2 * (m + 1));//桥
    std::vector<std::vector<int>>e_dcc(n + 1);//缩点的集合
    int tot = 0,times = 0;
    auto tarjan = [&](auto &&self,int u,int p) -> void
    {
        low[u] = dfn[u] = ++times;
        for (int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if (!dfn[v])
            {
                self(self,v,i);
                if(low[v] > dfn[u])bz[i] = bz[i ^ 1] = 1;
                low[u] = std::min(low[u], low[v]);
            }
            else if(i != (p ^ 1))
                low[u] = std::min(low[u], dfn[v]);
        }
    };
    auto dfs1 = [&](auto &&self,int u,int id) -> void
    {
        bel[u] = id;
        e_dcc[id].push_back(u);
```

```
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(bel[v] || bz[i])continue;
            self(self,v,id);
        }
    };
    for(int i = 1;i <= n;i++)
        if(!dfn[i])tarjan(tarjan,i,0);
    for(int i = 1;i <= n;i++)
        if(!bel[i])dfs1(dfs1,i,++tot);
}
```

点双连通分量V-DCC

```
std::vector<int>v_dcc[N];
int tot,times;
void solve()
{
    tot = times = 0;
    int n,m;
    std::cin >> n >> m;
    std::vector<std::vector<int>>g(n + 1);
    std::vector<int>low(n + 1),dfn(n + 1),cut(n + 1);
    for(int i = 1;i <= m;i++)
    {
        int u,v;
        std::cin >> u >> v;
        if(u == v)continue;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::stack<int>stk;
    auto tarjan = [&](auto &&self,int u,int rt) -> void
    {
        low[u] = dfn[u] = ++times;
        stk.push(u);
        if(g[u].empty())//孤立点
        {
            v_dcc[++tot].push_back(u);
            return;
        }
        int cnt = 0;//子树数量
        for(int v : g[u])
        {
            if(!dfn[v])
            {
                self(self,v,rt);
                low[u] = std::min(low[u],low[v]);
                if(low[v] >= dfn[u])//作为割点
                {
                    cnt++,tot++;
```

```
                    while(1)
                    {
                        int x = stk.top();stk.pop();
                        v_dcc[tot].push_back(x);
                        if(x == v)break;
                    }
                    v_dcc[tot].push_back(u);
                }
            }
            else low[u] = std::min(low[u],dfn[v]);
        }
        cut[u] = cnt > (u == rt);
    };
    for(int i = 1;i <= n;i++)
        if(!dfn[i])tarjan(tarjan,i,i);
    std::cout << tot << std::endl;
    for(int i = 1;i <= tot;i++)
    {
        int len = v_dcc[i].size();
        std::cout << len << " ";
        for(int j = 0;j < len;j++)
            std::cout << v_dcc[i][j] << " \n"[j == len - 1];
    }
}
```

割点

```
std::vector<int>cut(n + 1),low(n + 1),dfn(n + 1);
int times = 0;
auto tarjan = [&](auto &&self,int u,int rt) -> void
{
    dfn[u] = low[u] = ++times;
    int cnt = 0;//子树数量
    for(int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if(!dfn[v])
        {
            self(self,v,rt);
            low[u] = std::min(low[u],low[v]);
            if(low[v] >= dfn[u])cnt++;
        }
        else
            low[u] = std::min(low[u],dfn[v]);
    }
    cut[u] = cnt > (rt == u);
};
```

割边

```
//初始化
```

```
cntqq = 1;

std::vector<bool>bz(2 * (m + 1));//从编号 2 开始
std::vector<int>low(n + 1),dfn(n + 1);
int times = 0;
//p代表到达u的边编号
auto tarjan = [&](auto &&self,int u,int p) -> void
{
    low[u] = dfn[u] = ++times;
    for (int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if (!dfn[v])
        {
            self(self,v,i);
            if(low[v] > dfn[u])bz[i] = bz[i ^ 1] = 1;
            low[u] = std::min(low[u], low[v]);
        }
        else if(i != (p ^ 1))//不走反向边
            low[u] = std::min(low[u], dfn[v]);
    }
};
```

二分图判断(判奇环)

```
vector<bool>vis(n + 1);
vector<int>col(n + 1);
auto dfs = [&](auto &&self,int u) -> bool
{
    vis[u] = true;
    for(int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if(vis[v])
        {
            if(col[u] == col[v])return false;
        }
        else
        {
            col[v] = col[u] ^ 1;
            if(!self(self,v))return false;
        }
    }
    return true;
};
auto check = [&]() -> bool
{
    for(int i = 1;i <= n;i++)
        if(!vis[i])
        {
            col[i] = 0;
            if(!dfs(dfs,i))return false;//是奇环(不是二分图)
```

```
        }
    return true;
}
```

最大二分图匹配

```
//匈牙利算法
bool vis[N][N],ask[N];
int matched[N];
int n, m, e,cnt = 0;
inline bool found(int x)
{
    for (int i = 1; i <= m; i++)
    {
        if (vis[x][i])
        {
            if (ask[i])continue;
            ask[i] = 1;
            if (!matched[i] || found(matched[i]))
            {
                matched[i] = x;
                return true;
            }
        }
    }
    return false;
}

void match()
{
    for (int i = 1; i <= n; i++)
    {
        memset(ask, 0, sizeof ask);
        if (found(i))
            cnt++;
    }
}
void solve()
{
    cin >> n >> m >> e;
    for (int i = 1; i <= e; i++)
    {
        int u, v;
        cin >> u >> v;
        vis[u][v] = 1;
    }
    match();
    cout << cnt << endl;
}
signed main() {
```

```
    IOS;
    int T = 1; //cin >> T;
    while (T--) {
        solve();
    }
}
//dinic算法(单位容量下，nsqrt的复杂度，优于匈牙利)
源点指向左部连边
右部指向汇点连边
左部指向右部连边
跑出来的最大流也就是最大匹配
```

最大流

```
//普通 n * nm,单位容量 m * sqrt(n)
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
    };
    int n,cntqq,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dep;
    Dinic(int n):n(n),cntqq(0)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dep.resize(n + 1);
    }
    void add(int u,int v,int cap)
    {
        e.push_back({heads[u],v,cap,0});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0});
        heads[v] = cntqq++;
    };
    bool bfs()
    {
        std::fill(dep.begin(),dep.end(),0);
        std::queue<int>que;
        que.push(s);
        dep[s] = 1;
        while(!que.empty())
        {
            int u = que.front();que.pop();
            for(int i = heads[u];~i;i = e[i].nx)
```

```
            {
                int v = e[i].to;
                if(dep[v] || e[i].cap == e[i].flow)continue;
                dep[v] = dep[u] + 1;
                que.push(v);
            }
        }
        return dep[t];
    }
    int dfs(int u,int flow)
    {
        if(u == t)return flow;
        int res = 0;
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res);
            int tf = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res += tf;
            if(res == flow)break;
        }
        return res;
    }
    int mxflow(int S,int T)
    {
        s = S,t = T;
        int ans = 0;
        while(bfs())
        {
            cur = heads;
            ans += dfs(S,INF);
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
    for(int i = 1;i <= m;i++)
    {
        int u,v,w;
        std::cin >> u >> v >> w;
        dinic.add(u,v,w);
    }
    std::cout << dinic.mxflow(s,t) << "\n";
}
```

最小费用最大流

```cpp
//无初始负环处理，无势能优化,O(Fnm)
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
        int co;
    };
    int n,cntqq,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dis,dep;
    Dinic(int n):n(n),cntqq(0)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dis.resize(n + 1);
        dep.resize(n + 1);
    }
    void add(int u,int v,int cap,int co)
    {
        e.push_back({heads[u],v,cap,0,co});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0,-co});
        heads[v] = cntqq++;
    };
    bool spfa()
    {
        std::fill(dis.begin(),dis.end(),INF);
        std::fill(dep.begin(),dep.end(),INF);
        std::vector<bool>vis(n + 1);
        dis[s] = dep[s] = 0;
        vis[s] = 1;
        std::queue<int>q;
        q.push(s);
        while(!q.empty())
        {
            int u = q.front();q.pop();
            vis[u] = 0;
            for(int i = heads[u];~i;i = e[i].nx)
            {
                if(e[i].cap <= e[i].flow)continue;
                int v = e[i].to;
                int d = dis[u] + e[i].co;
                if(dis[v] > d || (dis[v] == d && dep[v] > dep[u] + 1))
                {
                    dis[v] = d;
                    dep[v] = dep[u] + 1;
```

```
                    if(!vis[v])
                    {
                        q.push(v);
                        vis[v] = 1;
                    }
                }
            }
        }
        return dis[t] != INF;
    }
    pii dfs(int u,int flow)
    {
        if(u == t)return {flow,0};
        pii res{0,0};
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(dis[v] != dis[u] + e[i].co)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res[0]);
            auto [tf,tc] = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res[0] += tf;
            res[1] += tc + tf * e[i].co;
            if(res[0] == flow)break;
        }
        return res;
    }
    pii mxflow(int S,int T,int li = INF)
    {
        s = S,t = T;
        if(s == t)return {0,0};
        pii ans{0,0};
        while(ans[0] < li && spfa())
        {
            cur = heads;
            auto [f,c] = dfs(s,li - ans[0]);
            ans[0] += f;
            ans[1] += c;
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
    for(int i = 1;i <= m;i++)
    {
```

```
        int u,v,w,c;
        std::cin >> u >> v >> w >> c;
        dinic.add(u,v,w,c);
    }
    auto [f,c] = dinic.mxflow(s,t);
    std::cout << f << " " << c << "\n";
}
```

```
//负环处理,势能优化 + Dijkstra
// O(Knm + F(n + m)log(m))
//K为消环次数，F为最大流量
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
        int co;
    };
    int cntqq,n,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dis,dep,h;
    Dinic(int n):cntqq(0),n(n)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dis.resize(n + 1);
        dep.resize(n + 1);
        h.assign(n + 1,0);
    }
    void add(int u,int v,int cap,int co)
    {
        e.push_back({heads[u],v,cap,0,co});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0,-co});
        heads[v] = cntqq++;
    }
    int init()
    {
        int res = 0;
        std::vector<int>pre(n + 1);
        while(1)
        {
            std::fill(h.begin(),h.end(),0);
            std::fill(pre.begin(),pre.end(),-1);
            int x = -1;
            for(int i = 1;i <= n;i++)
            {
                x = -1;
                for(int j = 0;j < cntqq;j++)
```

```
                {
                    if(e[j].cap <= e[j].flow)continue;
                    int u = e[j ^ 1].to,v = e[j].to;
                    if(h[v] > h[u] + e[j].co)
                    {
                        h[v] = h[u] + e[j].co;
                        pre[v] = j;
                        x = v;
                    }
                }
                if(x == -1)break;
            }
            if(x == -1)break;

            for(int i = 1;i <= n;i++)
                x = e[pre[x] ^ 1].to;

            int v = x,f = INF;
            do
            {
                int id = pre[v];
                f = std::min(f,e[id].cap - e[id].flow);
                v = e[id ^ 1].to;
            }while(v != x);
            v = x;
            do
            {
                int id = pre[v];
                e[id].flow += f;
                e[id ^ 1].flow -= f;
                res += f * e[id].co;
                v = e[id ^ 1].to;
            }while(v != x);
        }
        return res;
    }
    bool dij()
    {
        std::fill(dis.begin(),dis.end(),INF);
        std::fill(dep.begin(),dep.end(),INF);
        std::priority_queue<piii,std::vector<piii>,std::greater<piii>>q;
        dis[s] = dep[s] = 0;
        q.push({0,0,s});
        while(!q.empty())
        {
            auto [d,k,u] = q.top();q.pop();
            if(d != dis[u] || k != dep[u])continue;
            for(int i = heads[u];~i;i = e[i].nx)
            {
                if(e[i].cap <= e[i].flow)continue;
                int v = e[i].to;
                int nd = d + e[i].co + h[u] - h[v];
                if(dis[v] > nd || (dis[v] == nd && dep[v] > k + 1))
```

```
                {
                    dis[v] = nd;
                    dep[v] = k + 1;
                    q.push({dis[v],dep[v],v});
                }
            }
        }
        if(dis[t] == INF)return false;
        for(int i = 1;i <= n;i++)
            if(dis[i] != INF)h[i] += dis[i];
        return true;
    }
    pii dfs(int u,int flow)
    {
        if(u == t)return {flow,0};
        pii res{0,0};
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(h[v] != h[u] + e[i].co)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res[0]);
            auto [tf,tc] = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res[0] += tf;
            res[1] += tc + tf * e[i].co;
            if(res[0] == flow)break;
        }
        return res;
    }
    pii mxflow(int S,int T,int li = INF)
    {
        s = S,t = T;
        if(s == t)return {0,0};
        pii ans{0,init()};
        while(ans[0] < li && dij())
        {
            cur = heads;
            auto [f,c] = dfs(s,li - ans[0]);
            ans[0] += f;
            ans[1] += c;
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
```

```
    for(int i = 1;i <= m;i++)
    {
        int u,v,w,c;
        std::cin >> u >> v >> w >> c;
        dinic.add(u,v,w,c);
    }
    auto [f,c] = dinic.mxflow(s,t);
    std::cout << f << " " << c << "\n";
}
```

欧拉回路/路径

```
//记得判连通
//两种都是对于字典序最小的欧拉回路/路径的输出
//对于无向图
int n,m;
cin >> n >> m;
vector<vector<PA>>g(n + 1);
vector<int>du(n + 1),heads(n + 1);
vector<bool>vis(m + 1);
for(int i = 1;i <= m;i++)
{
    int u,v;
    cin >> u >> v;
    g[u].push_back({v,i});
    g[v].push_back({u,i});
    du[u]++;
    du[v]++;
}
int st = 0,ed = 0;
for(int i = 1;i <= n;i++)
{
    if(du[i] & 1)
    {
        if(!st)st = i;
        else if(!ed)ed = i;
        else
        {
            cout << "No" << endl;
            return;
        }
    }
}
st = st ? st : 1;
for(int i = 1;i <= n;i++)
    sort(g[i].begin(),g[i].end());
stack<int>stk;
auto dfs = [&](auto &&self,int x) -> void
{
    for(int &i = heads[x];i < g[x].size();)
    {
        auto [v,op] = g[x][i++];
```

```
        if(vis[op])continue;
        vis[op] = 1;
        self(self,v);
    }
    stk.push(x);
};
dfs(dfs,st);
while(!stk.empty())
{
    cout << stk.top() << " ";
    stk.pop();
}
cout << endl;
//对于有向图
int n,m;
cin >> n >> m;
vector<vector<int>>g(n + 1);
vector du(n + 1,vector<int>(2));
vector<int>heads(n + 1);
for(int i = 1;i <= m;i++)
{
    int u,v;
    cin >> u >> v;
    g[u].push_back(v);
    du[v][0]++;
    du[u][1]++;
}
int st = 0,ed = 0;
bool op = 0;
for(int i = 1;i <= n;i++)
{
    if(du[i][0] == du[i][1])continue;
    else if(du[i][1] - du[i][0] == 1)
    {
        if(!st)st = i;
        else op = 1;
    }
    else if(du[i][0] - du[i][1] == 1)
    {
        if(!ed)ed = i;
        else op = 1;
    }
    else op = 1;
}
if(op)
{
    cout << "No" << endl;
    return;
}
for(int i = 1;i <= n;i++)
    sort(g[i].begin(),g[i].end());
st = st ? st : 1;
stack<int>stk;
```

```
auto dfs = [&](auto && self,int x) -> void
{
    for(int &i = heads[x];i < g[x].size();)
    {
        int v = g[x][i++];
        self(self,v);
    }
    stk.push(x);
};
dfs(dfs,st);
while(!stk.empty())
{
    cout << stk.top() << " ";
    stk.pop();
}
cout << endl;
```

点分治

```
//求询问的点对距离是否存在，点分治本体nlogn

std::vector<int>siz(n + 1),vis(n + 1),mx(n + 1);//找重心 + 删点
//连通块大小
auto get_siz = [&](auto &&self,int u,int f) -> int
{
    int sum = 1;
    for(auto [v,w] : g[u])
    {
        if(vis[v] || v == f)continue;
        sum += self(self,v,u);
    }
    return sum;
};
//重心为根
auto get_rt = [&](auto &&self,int u,int f,int sum,int &rt) -> void
{
    mx[u] = 0;
    siz[u] = 1;
    for(auto [v,w] : g[u])
    {
        if(vis[v] || v == f)continue;
        self(self,v,u,sum,rt);
        mx[u] = std::max(mx[u],siz[v]);
        siz[u] += siz[v];
    }
    mx[u] = std::max(mx[u],sum - siz[u]);
    if(mx[u] < mx[rt])rt = u;
};
//根据题目具体情况来写
auto calc = [&](int u) -> void
{
    std::vector<int>S(1,0);
```

```
    exist[0] = 1;
    auto get_dis = [&](auto &&self,int now,int f,int dis,std::vector<int>&tp) -> void
    {
        if(dis >= N)return;//小剪枝

        tp.push_back(dis);
        for(int i = 0;i < m;i++)
            if(0 <= que[i] - dis && que[i] - dis < N && exist[que[i] - dis])
                ans[i] = 1;
        for(auto [v,w] : g[now])
        {
            if(v == f || vis[v])continue;
            self(self,v,now,dis + w,tp);
        }
    };
    for(auto [v,w] : g[u])
    {
        if(vis[v])continue;
        std::vector<int>tmp;
        get_dis(get_dis,v,u,w,tmp);
        for(int x : tmp)
        {
            if(x < N)
            {
                S.push_back(x);
                exist[x] = 1;
            }
        }
    }
    for(int x : S)
        if(x < N)exist[x] = 0;
};
auto divide = [&](auto &&self,int u) -> void
{
    int sum = get_siz(get_siz,u,0);
    int rt = 0;mx[rt] = INF;
    get_rt(get_rt,u,0,sum,rt);

    calc(rt);
    vis[rt] = 1;
    for(auto [v,w] : g[rt])
        if(!vis[v])
            self(self,v);
};
divide(divide,1);


```

树上启发式合并

```
void solve()
{
```

```cpp
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>col(n + 1);
    for(int i = 1;i <= n;i++)std::cin >> col[i];

    std::vector<int>siz(n + 1),son(n + 1),dfn(n + 1),ind(n + 1);
    int tot = 0;
    auto dfs1 = [&](auto &&self,int u,int fa) -> void
    {
        dfn[u] = ++tot;
        ind[tot] = u;
        siz[u] = 1;
        int zson = -1;
        for(int v : g[u])
        {
            if(v == fa)continue;
            self(self,v,u);
            siz[u] += siz[v];
            if(siz[v] > zson)
            {
                son[u] = v;
                zson = siz[v];
            }
        }
    };
    dfs1(dfs1,1,0);
    std::vector<int>cnt(n + 1),ans(n + 1);
    int sum = 0;
    auto dfs = [&](auto &&self,int u,int fa,bool fl) -> void
    {
        for(int v : g[u])
        {
            if(v == fa || v == son[u])continue;
            self(self,v,u,0);
        }
        if(son[u])self(self,son[u],u,1);
        if(!cnt[col[u]])sum++;
        cnt[col[u]]++;
        for(int v : g[u])
        {
            if(v == fa || v == son[u])continue;
            for(int i = dfn[v];i <= dfn[v] + siz[v] - 1;i++)
            {
                int pos = ind[i];
                if(!cnt[col[pos]])sum++;
```

```
                cnt[col[pos]]++;
            }
        }
        ans[u] = sum;
        if(fl)return;
        for(int i = dfn[u];i <= dfn[u] + siz[u] - 1;i++)
        {
            int pos = ind[i];
            cnt[col[pos]]--;
            if(!cnt[col[pos]])sum--;
        }
    };
    dfs(dfs,1,0,1);
    int m;std::cin >> m;
    while(m--)
    {
        int u;std::cin >> u;
        std::cout << ans[u] << "\n";
    }
}
```